

Agents

mod chat_agent_executor

FUNCTION	DESCRIPTION
<code>create_react_agent</code>	Creates an agent graph that calls tools in a loop until a stopping condition is met.

class AgentState deprecated

Bases: `TypedDict`

⚡ Deprecated

AgentState has been moved to `langchain.agents`. Please update your import to `from langchain.agents import AgentState`.

The state of the agent.

func create_react_agent deprecated

```
create_react_agent(
    model: str
    | LanguageModelLike
    | Callable[[StateSchema, Runtime[ContextT]], BaseChatModel]
    | Callable[[StateSchema, Runtime[ContextT]], Awaitable[BaseChatModel]]
    | Callable[
        [StateSchema, Runtime[ContextT]], Runnable[LanguageModelInput,
        BaseMessage]
    ]
    | Callable[
        [StateSchema, Runtime[ContextT],
        Awaitable[Runnable[LanguageModelInput, BaseMessage]],
    ],
    tools: Sequence[BaseTool | Callable | dict[str, Any] | ToolNode,
    *,
```

```

prompt: Prompt | None = None,
response_format: StructuredResponseSchema
| tuple[str, StructuredResponseSchema]
| None = None,
pre_model_hook: RunnableLike | None = None,
post_model_hook: RunnableLike | None = None,
state_schema: StateSchemaType | None = None,
context_schema: type[Any] | None = None,
checkpointers: Checkpointer | None = None,
store: BaseStore | None = None,
interrupt_before: list[str] | None = None,
interrupt_after: list[str] | None = None,
debug: bool = False,
version: Literal["v1", "v2"] = "v2",
name: str | None = None,
**deprecated_kwargs: Any,
) -> CompiledStateGraph

```



Deprecated



`create_react_agent` has been moved to `langchain.agents`. Please update your import to `from langchain.agents import create_agent`.

Creates an agent graph that calls tools in a loop until a stopping condition is met.

For more details on using `create_react_agent`, visit [Agents](#) documentation.

PARAMETER	DESCRIPTION
<code>model</code>	The language model for the agent. Supports static and dynamic model selection. • Static model: A chat model instance (e.g., <code>ChatOpenAI</code>) or string identifier (e.g., <code>"openai:gpt-4"</code>) • Dynamic model: A callable with signature <code>(state, runtime) -> BaseChatModel</code> that returns different models based on runtime context If the model has tools bound via <code>bind_tools</code> or other configurations, the return type should be a <code>Runnable[LanguageModelInput, BaseMessage]</code>. Coroutines are also supported, allowing for asynchronous model selection.

PARAMETER

DESCRIPTION

Dynamic functions receive graph state and runtime, enabling context-dependent model selection. Must return a `BaseChatModel` instance. For tool calling, bind tools using `.bind_tools()`. Bound tools must be a subset of the `tools` parameter.

 Dynamic model

```
from dataclasses import dataclass

@dataclass
class ModelContext:
    model_name: str = "gpt-3.5-turbo"

# Instantiate models globally
gpt4_model = ChatOpenAI(model="gpt-4")
gpt35_model = ChatOpenAI(model="gpt-3.5-turbo")

def select_model(state: AgentState, runtime:
Runtime[ModelContext]) -> ChatOpenAI:
    model_name = runtime.context.model_name
    model = gpt4_model if model_name == "gpt-4" else
gpt35_model
    return model.bind_tools(tools)
```

 Dynamic Model Requirements

Ensure returned models have appropriate tools bound via `.bind_tools()` and support required functionality. Bound tools must be a subset of those specified in the `tools` parameter.

TYPE: `str` | `LanguageModelLike` | `Callable[[StateSchema, Runtime[ContextT]], BaseChatModel]` | `Callable[[StateSchema, Runtime[ContextT]], Awaitable[BaseChatModel]]` | `Callable[[StateSchema, Runtime[ContextT]], Runnable[LanguageModelInput, BaseMessage]]` | `Callable[[StateSchema, Runtime[ContextT]], Awaitable[Runnable[LanguageModelInput, BaseMessage]]]`

`tools`

A list of tools or a `ToolNode` instance. If an empty list is provided, the agent will consist of a single LLM node without tool calling.

PARAMETER	DESCRIPTION
	TYPE: <code>Sequence[BaseTool Callable dict[str, Any]] ToolNode</code>
<code>prompt</code>	An optional prompt for the LLM. Can take a few different forms: • <code>str</code>: This is converted to a <code>SystemMessage</code> and added to the beginning of the list of messages in <code>state["messages"]</code>. • <code>SystemMessage</code>: this is added to the beginning of the list of messages in <code>state["messages"]</code>. • <code>Callable</code>: This function should take in full graph state and the output is then passed to the language model. • <code>Runnable</code>: This runnable should take in full graph state and the output is then passed to the language model. TYPE: <code>Prompt None</code> DEFAULT: <code>None</code>
<code>response_format</code>	An optional schema for the final agent output. If provided, output will be formatted to match the given schema and returned in the 'structured_response' state key. If not provided, <code>structured_response</code> will not be present in the output state. Can be passed in as: • An OpenAI function/tool schema, • A JSON Schema, • A TypedDict class, • A Pydantic class. • A tuple <code>(prompt, schema)</code>, where schema is one of the above. The prompt will be used together with the model that is being used to generate the structured response.  Important <code>response_format</code> requires the model to support <code>.with_structured_output</code>

PARAMETER	DESCRIPTION
	 Note The graph will make a separate call to the LLM to generate the structured response after the agent loop is finished. This is not the only strategy to get structured responses, see more options in this guide. TYPE: StructuredResponseSchema <code>tuple[str, StructuredResponseSchema]</code> None DEFAULT: None
<code>pre_model_hook</code>	An optional node to add before the <code>agent</code> node (i.e., the node that calls the LLM). Useful for managing long message histories (e.g., message trimming, summarization, etc.). Pre-model hook must be a callable or a runnable that takes in current graph state and returns a state update in the form of <pre># At least one of `messages` or `llm_input_messages` MUST be provided { # If provided, will UPDATE the `messages` in the state "messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES), ...], # If provided, will be used as the input to the LLM, # and will NOT UPDATE `messages` in the state "llm_input_messages": [...], # Any other state keys that need to be propagated ... }</pre>  Important At least one of <code>messages</code> or <code>llm_input_messages</code> MUST be provided and will be used as an input to the <code>agent</code> node. The rest of the keys will be added to the graph state.

PARAMETER	DESCRIPTION
	 Warning If you are returning <code>messages</code> in the pre-model hook, you should OVERWRITE the <code>messages</code> key by doing the following: <pre>{ "messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES), *new_messages] ...}</pre> TYPE: RunnableLike None DEFAULT: None
<code>post_model_hook</code>	An optional node to add after the <code>agent</code> node (i.e., the node that calls the LLM). Useful for implementing human-in-the-loop, guardrails, validation, or other post-processing. Post-model hook must be a callable or a runnable that takes in current graph state and returns a state update.  Note Only available with <code>version="v2"</code>. TYPE: RunnableLike None DEFAULT: None
<code>state_schema</code>	An optional state schema that defines graph state. Must have <code>messages</code> and <code>remaining_steps</code> keys. Defaults to <code>AgentState</code> that

PARAMETER	DESCRIPTION
	defines those two keys.  Note <code>remaining_steps</code> is used to limit the number of steps the react agent can take. Calculated roughly as <code>recursion_limit - total_steps_taken</code>. If <code>remaining_steps</code> is less than 2 and tool calls are present in the response, the react agent will return a final AI Message with the content "Sorry, need more steps to process this request.". No <code>GraphRecursionError</code> will be raised in this case. TYPE: <code>StateSchemaType</code> <code>None</code> DEFAULT: <code>None</code>
<code>context_schema</code>	An optional schema for runtime context. TYPE: <code>type[Any]</code> <code>None</code> DEFAULT: <code>None</code>
<code>checkpointer</code>	An optional checkpoint saver object. This is used for persisting the state of the graph (e.g., as chat memory) for a single thread (e.g., a single conversation). TYPE: <code>Checkpointter</code> <code>None</code> DEFAULT: <code>None</code>
<code>store</code>	An optional store object. This is used for persisting data across multiple threads (e.g., multiple conversations / users). TYPE: <code>BaseStore</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_before</code>	An optional list of node names to interrupt before. Should be one of the following: <code>"agent"</code>, <code>"tools"</code>. This is useful if you want to add a user confirmation or other interrupt before taking an action. TYPE: <code>list[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_after</code>	An optional list of node names to interrupt after. Should be one of the following: <code>"agent"</code>, <code>"tools"</code>.

PARAMETER	DESCRIPTION
	This is useful if you want to return directly or run additional processing on an output. TYPE: <code>list[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>debug</code>	A flag indicating whether to enable debug mode. TYPE: <code>bool</code> DEFAULT: <code>False</code>
<code>version</code>	Determines the version of the graph to create. Can be one of: <code>"v1"</code>: The tool node processes a single message. All tool calls in the message are executed in parallel within the tool node. <code>"v2"</code>: The tool node processes a tool call. Tool calls are distributed across multiple instances of the tool node using the Send API. TYPE: <code>Literal['v1', 'v2']</code> DEFAULT: <code>'v2'</code>
<code>name</code>	An optional name for the <code>CompiledStateGraph</code>. This name will be automatically used when adding ReAct agent graph to another graph as a subgraph node - particularly useful for building multi-agent systems. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>

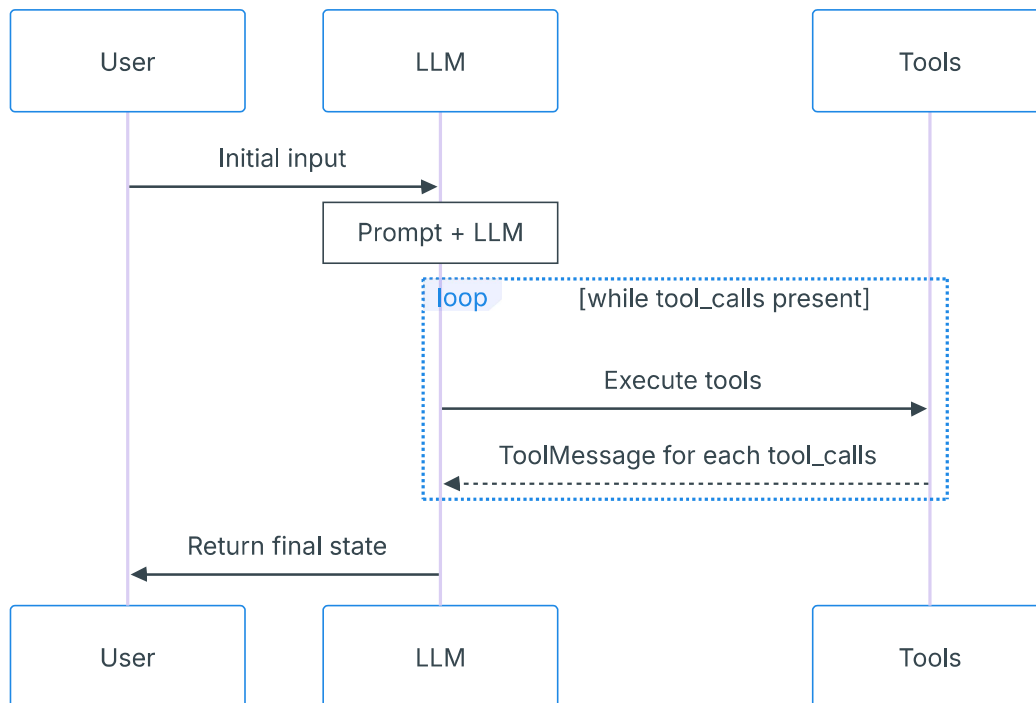


`config_schema` **Deprecated**

The `config_schema` parameter is deprecated in v0.6.0 and support will be removed in v2.0.0. Please use `context_schema` instead to specify the schema for run-scoped context.

RETURNS	DESCRIPTION
<code>CompiledStateGraph</code>	A compiled LangChain <code>Runnable</code> that can be used for chat interactions.

The "agent" node calls the language model with the messages list (after applying the prompt). If the resulting `AIMessage` contains `tool_calls`, the graph will then call the "tools". The "tools" node executes the tools (1 tool per `tool_call`) and adds the responses to the messages list as `ToolMessage` objects. The agent node then calls the language model again. The process repeats until no more `tool_calls` are present in the response. The agent then returns the full list of messages as a dictionary containing the key `'messages'`.



Example

```

from langgraph.prebuilt import create_react_agent

def check_weather(location: str) -> str:
    '''Return the weather forecast for the specified location.'''
    return f"It's always sunny in {location}"

graph = create_react_agent(
    "anthropic:claude-3-7-sonnet-latest",
    tools=[check_weather],
    prompt="You are a helpful assistant",
)
inputs = {"messages": [{"role": "user", "content": "what is the weather in sf"}]}
for chunk in graph.stream(inputs, stream_mode="updates"):
    print(chunk)

```

`class` ToolNode

Bases: `RunnableCallable`

A node for executing tools in LangGraph workflows.

Handles tool execution patterns including function calls, state injection, persistent storage, and control flow. Manages parallel execution, error handling.

Input Formats

1. Graph state with `messages` key that has a list of messages:
 - Common representation for agentic workflows
 - Supports custom messages key via `messages_key` parameter
2. **Message List:** `[AIMessage(..., tool_calls=[...])]`
 - List of messages with tool calls in the last `AIMessage`
3. **Direct Tool Calls:** `[{"name": "tool", "args": {...}, "id": "1", "type": "tool_call"}]`
 - Bypasses message parsing for direct tool execution
 - For programmatic tool invocation and testing

Output Formats

Output format depends on input type and tool behavior:

For Regular tools:

- Dict input → `{"messages": [ToolMessage(...)]}`
- List input → `[ToolMessage(...)]`

For Command tools:

- Returns `[Command(...)]` or mixed list with regular tool outputs
- `Command` can update state, trigger navigation, or send messages

PARAMETER	DESCRIPTION
<code>tools</code>	A sequence of tools that can be invoked by this node. Supports: • BaseTool instances: Tools with schemas and metadata • Plain functions: Automatically converted to tools with inferred schemas TYPE: <code>Sequence[BaseTool Callable]</code>
<code>name</code>	The name identifier for this node in the graph. Used for debugging and visualization. TYPE: <code>str</code> DEFAULT: <code>'tools'</code>
<code>tags</code>	Optional metadata tags to associate with the node for filtering and organization. TYPE: <code>List[str] None</code> DEFAULT: <code>None</code>
<code>handle_tool_errors</code>	Configuration for error handling during tool execution. Supports multiple strategies: • <code>True</code> : Catch all errors and return a <code>ToolMessage</code> with the default error template containing the exception details. • <code>str</code> : Catch all errors and return a <code>ToolMessage</code> with this custom error message string. • <code>type[Exception]</code> : Only catch exceptions with the specified type and return the default error message for it. • <code>tuple[type[Exception], ...]</code> : Only catch exceptions with the specified types and return default error messages for them. • <code>Callable[..., str]</code> : Catch exceptions matching the callable's signature and return the string result of calling it with the exception. • <code>False</code> : Disable error handling entirely, allowing exceptions to propagate. Defaults to a callable that:

PARAMETER	DESCRIPTION
	- Catches tool invocation errors (due to invalid arguments provided by the model) and returns a descriptive error message - Ignores tool execution errors (they will be re-raised) TYPE: <code>bool</code> <code>str</code> <code>Callable[..., str]</code> <code>type[Exception]</code> <code>tuple[type[Exception], ...]</code> DEFAULT: <code>_default_handle_tool_errors</code>
<code>messages_key</code>	The key in the state dictionary that contains the message list. This same key will be used for the output <code>ToolMessage</code> objects. Allows custom state schemas with different message field names. TYPE: <code>str</code> DEFAULT: <code>'messages'</code>

Examples:

Basic usage:

```
from langchain.tools import ToolNode
from langchain_core.tools import tool

@tool
def calculator(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

tool_node = ToolNode([calculator])
```

State injection:

```
from typing_extensions import Annotated
from langchain.tools import InjectedState

@tool
def context_tool(query: str, state: Annotated[dict, InjectedState]) -> str:
    """Some tool that uses state."""
    return f"Query: {query}, Messages: {len(state['messages'])}"

tool_node = ToolNode([context_tool])
```

Error handling:

```
def handle_errors(e: ValueError) -> str:
    return "Invalid input provided"

tool_node = ToolNode([my_tool], handle_tool_errors=handle_errors)
```

mod tool_node

Tool execution node for LangGraph workflows.

This module provides prebuilt functionality for executing tools in LangGraph.

Tools are functions that models can call to interact with external systems, APIs, databases, or perform computations.

The module implements design patterns for:

- Parallel execution of multiple tool calls for efficiency
- Robust error handling with customizable error messages
- State injection for tools that need access to graph state
- Store injection for tools that need persistent storage
- Command-based state updates for advanced control flow

Key Components:

- `ToolNode`: Main class for executing tools in LangGraph workflows
- `InjectedState`: Annotation for injecting graph state into tools
- `InjectedStore`: Annotation for injecting persistent store into tools
- `ToolRuntime`: Runtime information for tools, bundling together `state`, `context`, `config`, `stream_writer`, `tool_call_id`, and `store`
- `tools_condition`: Utility function for conditional routing based on tool calls

 Typical Usage 

```
from langchain_core.tools import tool
from langchain.tools import ToolNode

@tool
def my_tool(x: int) -> str:
    return f"Result: {x}"

tool_node = ToolNode([my_tool])
```

FUNCTION	DESCRIPTION
<code>tools_condition</code>	Conditional routing function for tool-calling workflows.

class `InjectedState`

Bases: `InjectedToolArg`

Annotation for injecting graph state into tool arguments.

This annotation enables tools to access graph state without exposing state management details to the language model. Tools annotated with `InjectedState` receive state data automatically during execution while remaining invisible to the model's tool-calling interface.

PARAMETER	DESCRIPTION
<code>field</code>	Optional key to extract from the state dictionary. If <code>None</code> , the entire state is injected. If specified, only that field's value is injected. This allows tools to request specific state components rather than processing the full state structure. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>

 Example

```

from typing import List
from typing_extensions import Annotated, TypedDict

from langchain_core.messages import BaseMessage, AIMessage
from langchain.tools import InjectedState, ToolNode, tool

class AgentState(TypedDict):
    messages: List[BaseMessage]
    foo: str

@tool
def state_tool(x: int, state: Annotated[dict, InjectedState]) -> str:
    '''Do something with state.'''
    if len(state["messages"]) > 2:
        return state["foo"] + str(x)
    else:
        return "not enough messages"

@tool
def foo_tool(x: int, foo: Annotated[str, InjectedState("foo")]) -> str:
    '''Do something else with state.'''
    return foo + str(x + 1)

node = ToolNode([state_tool, foo_tool])

tool_call1 = {"name": "state_tool", "args": {"x": 1}, "id": "1", "type":
"tool_call"}
tool_call2 = {"name": "foo_tool", "args": {"x": 1}, "id": "2", "type": "tool_call"}
state = {
    "messages": [AIMessage("", tool_calls=[tool_call1, tool_call2])],
    "foo": "bar",
}
node.invoke(state)

[
    ToolMessage(content="not enough messages", name="state_tool",
tool_call_id="1"),
    ToolMessage(content="bar2", name="foo_tool", tool_call_id="2"),
]

```

Note

- `InjectedState` arguments are automatically excluded from tool schemas presented to language models
- `ToolNode` handles the injection process during execution
- Tools can mix regular arguments (controlled by the model) with injected arguments (controlled by the system)
- State injection occurs after the model generates tool calls but before tool execution

METHOD	DESCRIPTION
<code>__init__</code>	Initialize the <code>InjectedState</code> annotation.

meth `__init__`

```
__init__(field: str | None = None) -> None
```

Initialize the `InjectedState` annotation.

class `InjectedStore`

Bases: `InjectedToolArg`

Annotation for injecting persistent store into tool arguments.

This annotation enables tools to access LangGraph's persistent storage system without exposing storage details to the language model. Tools annotated with `InjectedStore` receive the store instance automatically during execution while remaining invisible to the model's tool-calling interface.

The store provides persistent, cross-session data storage that tools can use for maintaining context, user preferences, or any other data that needs to persist beyond individual workflow executions.

Warning

`InjectedStore` annotation requires `langchain-core >= 0.3.8`

Example

```
from typing_extensions import Annotated
from langgraph.store.memory import InMemoryStore
from langchain.tools import InjectedStore, ToolNode, tool
```

```
@tool
def save_preference(
    key: str,
    value: str,
    store: Annotated[Any, InjectedStore()]
) -> str:
    """Save user preference to persistent storage."""
    store.put(("preferences",), key, value)
    return f"Saved {key} = {value}"

@tool
def get_preference(
    key: str,
    store: Annotated[Any, InjectedStore()]
) -> str:
    """Retrieve user preference from persistent storage."""
    result = store.get(("preferences",), key)
    return result.value if result else "Not found"
```

Usage with `ToolNode` and graph compilation:

```
from langgraph.graph import StateGraph
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
tool_node = ToolNode([save_preference, get_preference])

graph = StateGraph(State)
graph.add_node("tools", tool_node)
compiled_graph = graph.compile(store=store) # Store is injected automatically
```

Cross-session persistence:

```
# First session
result1 = graph.invoke({"messages": [HumanMessage("Save my favorite color as blue")]}))

# Later session - data persists
result2 = graph.invoke({"messages": [HumanMessage("What's my favorite color?")]}))
```

Note

- `InjectedStore` arguments are automatically excluded from tool schemas presented to language models
- The store instance is automatically injected by `ToolNode` during execution
- Tools can access namespaced storage using the store's get/put methods
- Store injection requires the graph to be compiled with a store instance
- Multiple tools can share the same store instance for data consistency

func `tools_condition`

```
tools_condition(  
    state: list[AnyMessage] | dict[str, Any] | BaseModel, messages_key: str =  
    "messages"  
) -> Literal["tools", "__end__"]
```

Conditional routing function for tool-calling workflows.

This utility function implements the standard conditional logic for ReAct-style agents: if the last `AIMessage` contains tool calls, route to the tool execution node; otherwise, end the workflow. This pattern is fundamental to most tool-calling agent architectures.

The function handles multiple state formats commonly used in LangGraph applications, making it flexible for different graph designs while maintaining consistent behavior.

PARAMETER	DESCRIPTION
<code>state</code>	The current graph state to examine for tool calls. Supported formats: - Dictionary containing a messages key (for <code>StateGraph</code>) - <code>BaseModel</code> instance with a messages attribute TYPE: <code>list[AnyMessage]</code> <code>dict[str, Any]</code> <code>BaseModel</code>
<code>messages_key</code>	The key or attribute name containing the message list in the state. This allows customization for graphs using different state schemas. TYPE: <code>str</code> DEFAULT: <code>'messages'</code>

RETURNS	DESCRIPTION
<code>Literal['tools', '__end__']</code>	Either <code>'tools'</code> if tool calls are present in the last <code>AIMessage</code> , or <code>'__end__'</code> to terminate the workflow. These are the standard routing destinations for tool-calling conditional edges.

RAISES	DESCRIPTION
<code>ValueError</code>	If no messages can be found in the provided state format.

Example

Basic usage in a ReAct agent:

```
from langgraph.graph import StateGraph
from langchain.tools import ToolNode
from langchain.tools.tool_node import tools_condition
from typing_extensions import TypedDict

class State(TypedDict):
    messages: list

graph = StateGraph(State)
graph.add_node("llm", call_model)
graph.add_node("tools", ToolNode([my_tool]))
graph.add_conditional_edges(
    "llm",
    tools_condition, # Routes to "tools" or "__end__"
    {"tools": "tools", "__end__": "__end__"},
)
```

Custom messages key:

```
def custom_condition(state):
    return tools_condition(state, messages_key="chat_history")
```

 Note

This function is designed to work seamlessly with `ToolNode` and standard LangGraph patterns. It expects the last message to be an `AIMessage` when tool calls are present, which is the standard output format for tool-calling language models.

class `ValidationNode` `deprecated`

Bases: `RunnableCallable`

 Deprecated

`ValidationNode` is deprecated. Please use `create_agent` from `langchain.agents` with custom tool error handling.

A node that validates all tools requests from the last `AIMessage`.

It can be used either in `StateGraph` with a `'messages'` key.

 Note

This node does not actually **run** the tools, it only validates the tool calls, which is useful for extraction and other use cases where you need to generate structured output that conforms to a complex schema without losing the original messages and tool IDs (for use in multi-turn conversations).

RETURNS

`Union[Dict[str, List[ToolMessage]],
Sequence[ToolMessage]]`

DESCRIPTION

A list of `ToolMessage` objects with the validated content or error messages.

Example

Example usage for re-prompting the model to generate a valid response:

```
from typing import Literal, Annotated
from typing_extensions import TypedDict

from langchain_anthropic import ChatAnthropic
from pydantic import BaseModel, field_validator

from langgraph.graph import END, START, StateGraph
from langgraph.prebuilt import ValidationNode
from langgraph.graph.message import add_messages

class SelectNumber(BaseModel):
    a: int

    @field_validator("a")
    def a_must_be_meaningful(cls, v):
        if v != 37:
            raise ValueError("Only 37 is allowed")
        return v

builder = StateGraph(Annotated[list, add_messages])
llm = ChatAnthropic(model="claude-3-5-haiku-latest").bind_tools([SelectNumber])
builder.add_node("model", llm)
builder.add_node("validation", ValidationNode([SelectNumber]))
builder.add_edge(START, "model")

def should_validate(state: list) -> Literal["validation", "__end__"]:
    if state[-1].tool_calls:
        return "validation"
    return END

builder.add_conditional_edges("model", should_validate)

def should_reprompt(state: list) -> Literal["model", "__end__"]:
    for msg in state[:-1]:
        # None of the tool calls were errors
        if msg.type == "ai":
            return END
        if msg.additional_kwargs.get("is_error"):
            return "model"
    return END

builder.add_conditional_edges("validation", should_reprompt)

graph = builder.compile()
res = graph.invoke(("user", "Select a number, any number"))
# Show the retry logic
for msg in res:
    msg.pretty_print()
```

mod interrupt**class** HumanInterruptConfig deprecatedBases: `TypedDict`**Deprecated**

HumanInterruptConfig has been moved to `langchain.agents.interrupt`. Please update your import to `from langchain.agents.interrupt import HumanInterruptConfig`.

Configuration that defines what actions are allowed for a human interrupt.

This controls the available interaction options when the graph is paused for human input.

ATTRIBUTE	DESCRIPTION
<code>allow_ignore</code>	Whether the human can choose to ignore/skip the current step
<code>allow_respond</code>	Whether the human can provide a text response/feedback
<code>allow_edit</code>	Whether the human can edit the provided content/state
<code>allow_accept</code>	Whether the human can accept/approve the current state

class ActionRequest deprecatedBases: `TypedDict`**Deprecated**

ActionRequest has been moved to `langchain.agents.interrupt`. Please update your import to `from langchain.agents.interrupt import ActionRequest`.

Represents a request for human action within the graph execution.

Contains the action type and any associated arguments needed for the action.

ATTRIBUTE	DESCRIPTION
<code>action</code>	The type or name of action being requested (e.g., "Approve XYZ action")
<code>args</code>	Key-value pairs of arguments needed for the action

`class` HumanInterrupt deprecated

Bases: `TypedDict`

 **Deprecated** 

HumanInterrupt has been moved to `langchain.agents.interrupt`. Please update your import to `from langchain.agents.interrupt import HumanInterrupt`.

Represents an interrupt triggered by the graph that requires human intervention.

This is passed to the `interrupt` function when execution is paused for human input.

ATTRIBUTE	DESCRIPTION
<code>action_request</code>	The specific action being requested from the human
<code>config</code>	Configuration defining what actions are allowed
<code>description</code>	Optional detailed description of what input is needed

**Example**

```
# Extract a tool call from the state and create an interrupt request
request = HumanInterrupt(
    action_request=ActionRequest(
        action="run_command", # The action being requested
        args={"command": "ls", "args": ["-l"]} # Arguments for the action
    ),
    config=HumanInterruptConfig(
        allow_ignore=True, # Allow skipping this step
        allow_respond=True, # Allow text feedback
        allow_edit=False, # Don't allow editing
        allow_accept=True # Allow direct acceptance
    ),
    description="Please review the command before execution"
)
# Send the interrupt request and get the response
response = interrupt([request])[0]
```

class HumanResponseBases: `TypedDict`

The response provided by a human to an interrupt, which is returned when graph execution resumes.

ATTRIBUTE	DESCRIPTION
<code>type</code>	The type of response: <code>'accept'</code>: Approves the current state without changes <code>'ignore'</code>: Skips/ignores the current step <code>'response'</code>: Provides text feedback or instructions <code>'edit'</code>: Modifies the current state/content TYPE: <code>Literal['accept', 'ignore', 'response', 'edit']</code>
<code>args</code>	The response payload: <code>None</code>: For ignore/accept actions <code>str</code>: For text responses <code>ActionRequest</code>: For edit actions with updated content

ATTRIBUTE	DESCRIPTION
	TYPE: None str ActionRequest